Université Grenoble Alpes
L3 Informatique

Année 2026-2027

Architectures logicielles et matérielles
Semaine 4 TP 2

Nous poursuivons ici l'utilisation comparée du polling (attente active) et d'interruptions lors de l'acquisition de données à partir du RNG (Random Number Generator).
Vous travaillerez ici avec les fichiers de la même archive qu'utilisée au TP précédent.

Dans ce deuxième TP, nous allons réaliser une **solution utilisant des interruptions.** Au lieu de scruter l'état du périphérique RNG pour repérer la disponibilté d'une nouvelle valeur produite, nous allons configurer le composant de telle façon qu'une ***interruption soit émise*** *chaque fois qu'un nouveau nombre aléatoire est disponible* (cf slide 39 du cours).

Question 1.
Rappeler le **principe général** d'une telle gestion d'interruptions : que se passera-t-il lors de la réception de l'interruption, et au retour du traitement de l'interruption ?

Préciser ce que doit faire en général le traitant d'interruption.

Et plus précisément, quelle **action** devrons-nous faire **ici** lors de la réception de chaque interruption ?

Ne pas *continuer si vous ne maîtrisez pas la réponse à cette question !...*

Nous utiliserons ici le fichier *firmware_interrupt.c*, renommer ce fichier *firmware.c*
Commencer par inclure dans le code fourni les fonctions *write_leds_int* et *sys_tick_handler* précédemment développées.

Question 2.
La documentation du RNG nous indique qu'il faut :

- *activer* l'interruption en positionnant à 1 le bit IE du registre RNG_CR.
Attention, une interruption sera générée lorsqu'un nouveau nombre aléatoire est disponible ou lorsqu'une erreur s'est produite. Il peut s'agir soit d'un problème avec la graine (seed error) soit d'un problème provenant de l'horloge (clock error), nous approfondirons plus loin.
- *activer* la génération de nombres aléatoires en positionnant à 1 le bit RNGEN du registre RNG_CR.

Rappeler le **lien** entre la représentation ci-dessous de ce registre **RNG_CR** et les constantes **`RNG_CR_RNGEN`** et **`RNG_CR_IE`** données en Annexe. Quelles valeurs représentent exactement ces 2 constantes ?

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved												IE	RNGEN	Reserved	
												rw	rw		

Question 3.
Rappeler par quel appel à la fonction **`nvic_enable_irq`** l'interruption du RNG sera **activée au niveau du NVIC** (voir cours, réponse au slide 31).

Définir la **fonction**

```
void init_rng();
```

qui réalise toutes les opérations pour l'activation de l'interruption.
(attention, la fonction **`nvic_enable_irq`** sera la seule fonction de la libopencm3 utilisée ici)

Question 4.
Enfin, il nous faudra de plus prendre en compte la gestion d'erreurs.
La documentation précise que lors de chaque interruption il sera nécessaire de vérifier qu'aucune erreur ne s'est produite, et que la donnée est bien disponible.
Pour vérifier *qu'il n'y a pas eu d'erreur*, il faut s'assurer que les bits SEIS et CEIS du registre RNG_SR sont à 0.
Pour vérifier *qu'un nouveau nombre aléatoire est bien disponible*, il faut s'assurer que le bit DRDY du registre RNG_SR est à 1 (noter que ce bit est automatiquement remis à 0 à la lecture de la donnée).

Faire le **lien** entre la représentation ci-dessous de ce registre **RNG_SR** et les constantes **`RNG_SR_SEIS`**, **`RNG_SR_CEIS`** et **`RNG_SR_DRDY`** données en Annexe. Quelles valeurs représentent exactement ces 3 constantes ?

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved									SEIS	CEIS	Reserved		SECS	CECS	DRDY
									rc_w0	rc_w0			r	r	r

Ecrire les 3 **fonctions** suivantes (ici aussi, on n'utilisera aucune fonction de la libopencm3) :

```
int check_no_seed_error();    // renvoie 1 si pas de seed error, 0 sinon
int check_no_clock_error();   // renvoie 1 si pas de clock error, 0 sinon
int check_data_ready();     // renvoie 1 si donnée disponible, 0 sinon
```

/!\ Il est important de noter que, lorsqu'une donnée valide est présente, la lecture de cette donnée provoque automatiquement une remise à 0 du bit DRDY, ainsi que l'annulation de l'interruption.

Question 5.
Il reste maintenant à écrire le **gestionnaire d'interruptions**, qui récupère le nouveau nombre aléatoire et le place dans la variable **`rnd`**, pourvu qu'il n'y ait pas eu d'erreur et que le nombre soit bien disponible.
Attention, la documentation signale aussi qu'en cas d'erreur, il faut* :
- remettre à 0 le bit CEIS de RNG_SR
- remettre à 0 le bit SEIS de RNG_SR
- remettre à 1 le bit RNGEN de RNG_CR (pour réinitialiser et redémarrer le RNG)

Question 6.
Enfin, comme au TP précédent, nous voulons faire en sorte d'avoir une séquence d'allumages aléatoires des leds, en **reprovoquant les appels à la fonction d'allumage périodiquement**, grâce au périphérique *systick*. Prenez donc garde à bien avoir inclus dans votre code les fonctions nécessaires pour son utilisation (cf TP précédent).

Est-il nécessaire de modifier le ***main*** fourni ? Si oui, expliquez et modifiez comme cela vous semble nécessaire; si non, expliquez pourquoi et comment l'application va se comporter.

Vous pourrez tester de façon similaire au TP précédent.

Question optionnelle. Mais combien de fois un nouveau nombre aléatoire a-t-il été produit entre chaque appel au gestionnaire d'interruptions du *systick* ?
Pour observer cela, il suffit d'initialiser une variable globale entière **`count_rng_interrupts`** à 0, de l'incrémenter à chaque interruption en provenance du RNG, et de la remettre à 0 à chaque interruption en provenance du *systick* !
Après avoir fait cela, cross-compiler et charger l'exécutable, placer un breakpoint sur le gestionnaire d'interruptions du *systick*, et continuer de breakpoint en breakpoint, en faisant afficher la valeur de **`count_rng_interrupts`**. Quelle valeur trouvez-vous ?

---

* Si cela se produit, vous pouvez de plus par exemple faire allumer toutes les leds, par : `write_leds_int(15);`

Annexe

```
/* This file is part of the libopencm3 project.
 *
 * This library is free software: you can redistribute it and/or modify
 * it under the terms of the GNU Lesser General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This library is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
 * GNU Lesser General Public License for more details.
 *
 * You should have received a copy of the GNU Lesser General Public License
 * along with this library.  If not, see <http://www.gnu.org/licenses/>.
 */

/** @cond */
#ifdef LIBOPENCM3_RNG_H
/** @endcond */
#ifndef LIBOPENCM3_RNG_COMMON_F24_H
#define LIBOPENCM3_RNG_COMMON_F24_H

// #define MMIO32(addr)        (*(volatile uint32_t *)(addr))

/* --- Random number generator registers ---------------------------------- */

/* Control register */
#define RNG_CR                  MMIO32(RNG_BASE + 0x00)

/* Status register */
#define RNG_SR                  MMIO32(RNG_BASE + 0x04)

/* Data register */
#define RNG_DR                  MMIO32(RNG_BASE + 0x08)

/* --- RNG_CR values ------------------------------------------------------ */

/* RNG ENABLE */
#define RNG_CR_RNGEN            (1 << 2)

/* RNG interrupt enable */
#define RNG_CR_IE               (1 << 3)

/* --- RNG_SR values ------------------------------------------------------ */

/* Data ready */
#define RNG_SR_DRDY             (1 << 0)

/* Clock error current status */
#define RNG_SR_CECS             (1 << 1)

/* Seed error current status */
#define RNG_SR_SECS             (1 << 2)

/* Clock error interrupt status */
#define RNG_SR_CEIS             (1 << 5)

/* Seed error interrupt status */
#define RNG_SR_SEIS             (1 << 6)

#endif
/** @cond */
#else
#warning "rng_common_f24.h should not be included explicitly, only via rng.h"
#endif
/** @endcond */
```